



ESCUELA DE INGENIERÍA DE FUENLABRADA

GRADO EN INGENIERÍA TELEMÁTICA

TRABAJO FIN DE GRADO

MAPAS DE TERRENO PARA REALIDAD EXTENDIDA

Autor : José Fuhui Pérez Contreras

Tutor : Dr. David Moreno Lumbreras

Curso académico 2024/2025



©2025 José Fuhui Pérez Contreras

Algunos derechos reservados

Este documento se distribuye bajo la licencia
“Atribución-CompartirIgual 4.0 Internacional” de Creative Commons,

disponible en

<https://creativecommons.org/licenses/by-sa/4.0/deed.es>

*Dedicado a
mis padres*

Agradecimientos

Quiero agradecer a todas las personas que me han ayudado a lo largo de mi carrera, y en especial a mis amigos de clase y a mi tutor, David, por su apoyo y ayuda en la realización de este trabajo. También quiero agradecer a mis padres por su apoyo incondicional y por haberme dado la oportunidad de estudiar en esta universidad.

Resumen

Este proyecto tiene como objetivo la creación de un componente en tres dimensiones compatible con la realidad virtual. El componente utiliza *APIS* de *Mapbox* y *OpenStreetMap* para la creación de mapas de terreno en tres dimensiones. Se pretende que el componente pueda ser utilizado en aplicaciones de realidad virtual y que sea fácil de integrar en otros proyectos.

El componente se ha desarrollado utilizando *JavaScript* y *HTML*, apoyándose en los entornos de trabajo de *Node.js*, *A-Frame* (y, por tanto, *WebGL* y *Three.js*) y *BabiaXR*. La intención es que esté disponible en la web para que cualquier persona pueda utilizarlo y contribuir a su desarrollo.

Los mapas son una herramienta muy útil para la visualización de datos y la navegación en entornos tridimensionales. Durante toda la historia se han valorado los mapas como una herramienta fundamental para la exploración y el conocimiento del mundo. En la actualidad, con el auge de la realidad virtual y la realidad aumentada, los mapas han cobrado una nueva dimensión y se están convirtiendo en una herramienta esencial para la creación de experiencias inmersivas. Es por ello que pienso que este proyecto es relevante y puede tener un impacto significativo en el campo de la realidad virtual y la visualización de datos.

Summary

This project aims to create a three-dimensional component compatible with virtual reality. The component uses APIs from *Mapbox* and *OpenStreetMap* to generate three-dimensional terrain maps. The goal is for the component to be usable in virtual reality applications and to be easy to integrate into other projects.

The component has been developed using *JavaScript* and *HTML*, relying on the *Node.js*, *A-Frame* (and therefore *WebGL* and *Three.js*) and *BabiaXR* frameworks. The intention is to make it available on the web so that anyone can use it and contribute to its development.

Maps are a very useful tool for data visualization and navigation in three-dimensional environments. Throughout history, maps have been valued as a fundamental tool for exploration and understanding of the world. Nowadays, with the rise of virtual reality and augmented reality, maps have taken on a new dimension and are becoming an essential tool for creating immersive experiences. That is why I believe this project is relevant and may have a significant impact in the field of virtual reality and data visualization.

Índice general

1. Introducción	1
1.1. Contexto	2
1.2. Estructura de la memoria	3
2. Objetivos	5
2.1. Objetivo general	5
2.2. Objetivos específicos	6
2.3. Planificación temporal	6
3. Estado del arte	9
3.1. Babia-XR	9
3.1.1. A-Frame	10
3.1.2. Three.js	11
3.1.3. WebXR	12
3.2. HTML	12
3.3. JavaScript y Node.js	13
3.3.1. Node.js y herramientas asociadas	14
3.4. APIs de Mapbox y <i>OpenStreetMap</i>	14
3.5. Gafas de Realidad Virtual: Meta Quest 3	15
3.6. Editor de Código: Visual Studio Code	16
3.7. Apoyo de herramientas de inteligencia artificial	16
4. Diseño e implementación	19
4.1. Proceso de formación	20
4.2. Arquitectura general	25

5. Experimentos y validación	37
6. Resultados	39
7. Conclusiones	41
7.1. Consecución de objetivos	41
7.2. Aplicación de lo aprendido	41
7.3. Lecciones aprendidas	42
7.4. Trabajos futuros	42
A. Manual de usuario	43
A.1. Uso del componente <code>babiaxr</code>	43
Bibliografía	47

Índice de figuras

2.1. Diagrama de Gantt del TFG.	8
3.1. Sitio web oficial de <i>A-Frame</i>	11
3.2. Código HTML del componente.	13
3.3. Código fuente en <i>JavaScript</i> del componente.	14
3.4. Sitio web oficial de <i>OpenStreetMap</i>	15
3.5. Gafas prestadas por la universidad.	16
4.1. Función para generar imagen en escala de grises	27
4.2. Creación de la malla mediante una función flecha	30

Capítulo 1

Introducción

En los últimos años, el desarrollo de tecnologías inmersivas como la realidad virtual (VR) y la realidad aumentada (AR) ha propiciado nuevas formas de interactuar con la información digital. En este contexto, los mapas tridimensionales se presentan como una herramienta clave para enriquecer experiencias inmersivas, facilitar la navegación en entornos complejos y mejorar la visualización de datos geoespaciales. Este proyecto se enmarca en dicha línea de innovación, proponiendo la creación de un componente web en tres dimensiones que permita representar mapas de terreno realistas y compatibles con dispositivos de realidad virtual.

El componente desarrollado hace uso de diversas tecnologías modernas del ecosistema web, como *A-Frame*, *WebGL* y *Three.js*, facilitando la creación de entornos 3D interactivos en navegadores estándar. A su vez, se apoya en servicios de terceros como *Mapbox* y *OpenStreetMap*, que proporcionan datos geográficos y texturas necesarias para representar el terreno con un alto nivel de detalle. Todo ello se integra mediante *JavaScript*, *HTML* y herramientas de desarrollo como *Node.js*, dentro del ecosistema de la plataforma *BabiaXR*, orientada a facilitar el desarrollo de experiencias inmersivas en la web.

Uno de los principales objetivos del proyecto es que el componente resultante sea modular, reutilizable y accesible para otros desarrolladores o investigadores que quieran incorporarlo en sus propias aplicaciones. Por este motivo, se ha priorizado su disponibilidad en la web y la posibilidad de que terceros puedan contribuir a su mejora.

Además del reto técnico, el proyecto plantea una reflexión sobre el papel actual de los mapas en la era digital. A lo largo de la historia, los mapas han sido instrumentos

fundamentales para la exploración, la planificación y la representación del mundo. Hoy, con el auge de las tecnologías inmersivas, su potencial se amplía hacia nuevas formas de interacción, análisis y visualización de datos. En este sentido, este proyecto busca aportar una herramienta útil y abierta que contribuya a la evolución del uso de los mapas en entornos virtuales.

1.1. Contexto

Los mapas han sido históricamente una herramienta fundamental para representar el territorio y facilitar la navegación, la planificación y la toma de decisiones. En la actualidad, su importancia ha ido en aumento gracias a los avances tecnológicos que permiten su digitalización, interacción y visualización tridimensional en tiempo real.

En el ámbito de la Ingeniería Telemática, los mapas no solo se utilizan como elementos visuales, sino también como interfaces que permiten acceder, consultar y analizar grandes volúmenes de datos georreferenciados. Esta relación directa con las bases de datos espaciales establece un nexo claro con asignaturas cursadas durante el grado, como *Laboratorio de Bases de Datos*, donde se han trabajado conceptos fundamentales como la estructuración eficiente de datos, su recuperación y la optimización de consultas.

El uso de mapas tridimensionales y tecnologías asociadas como *WebXR* abre un abanico de posibilidades en diferentes áreas. Desde la educación inmersiva y la visualización científica, hasta la planificación urbana, el turismo o la simulación de entornos complejos. Al permitir representar información en contextos espaciales, los mapas se convierten en una poderosa herramienta de comunicación y análisis.

Este proyecto surge del interés por combinar estas capacidades con tecnologías modernas de visualización en entornos web, apoyado además por el hecho de que *Babia XR*, la herramienta empleada, ha sido desarrollada por docentes e investigadores de la Universidad Rey Juan Carlos. Este hecho no solo facilita el acceso al entorno de trabajo, sino que también refuerza el valor académico del proyecto al fomentar la colaboración con desarrollos propios del entorno universitario.

1.2. Estructura de la memoria

Esta memoria se estructura de la siguiente manera:

- En este capítulo inicial se presenta el contexto general del proyecto, los antecedentes que lo motivan y los objetivos que persigue. Además, se describe la estructura del documento para orientar al lector.
- El capítulo 2 expone los objetivos generales y específicos del proyecto, junto con la planificación temporal diseñada para su desarrollo.
- A continuación, en el capítulo 3 se analiza el estado del arte, incluyendo una revisión de tecnologías afines, soluciones previas y herramientas relevantes para el desarrollo de experiencias interactivas en realidad virtual.
- El capítulo 4 aborda el diseño y la implementación del componente desarrollado, con especial atención a su arquitectura, funcionalidades principales y decisiones técnicas adoptadas durante el proceso.
- En el capítulo 5 se describen los experimentos realizados y la validación del prototipo, incluyendo los criterios utilizados para evaluar su funcionamiento.
- El capítulo 6 recoge los resultados obtenidos tras la fase de pruebas, valorando en qué medida se han alcanzado los objetivos planteados.
- Por último, el capítulo 7 recoge las conclusiones generales del trabajo, analiza el grado de cumplimiento de los objetivos, reflexiona sobre el aprendizaje adquirido y propone posibles líneas de trabajo futuro.
- Como material complementario, se incluye un apéndice con un manual de usuario, así como la bibliografía utilizada en la elaboración del proyecto.

Capítulo 2

Objetivos

2.1. Objetivo general

El objetivo principal de este proyecto es desarrollar un componente que permita la visualización e interacción con mapas tridimensionales en un entorno de realidad virtual accesible directamente desde un navegador web. La meta es que cualquier usuario pueda acceder a la experiencia sin necesidad de instalar software adicional, accediendo a un entorno inmersivo y navegable desde la propia aplicación web.

Para ello, se ha trabajado con tecnologías web avanzadas como *A-Frame* y *WebXR*, que permiten renderizar escenas tridimensionales y gestionar la interacción en entornos de realidad virtual utilizando únicamente *HTML* y *JavaScript*. Además, se hace uso de servicios de mapas y datos de elevación para representar de forma realista el terreno, mejorando la sensación de inmersión.

Uno de los aspectos clave del proyecto es la ****interacción en tiempo real****, es decir, que el usuario pueda moverse por el mapa, hacer zoom, cambiar la textura o incluso modificar parámetros del terreno sin abandonar la aplicación. Esta capacidad convierte al componente en una herramienta versátil que puede ser aplicada en múltiples contextos, desde la enseñanza y la divulgación científica, hasta el urbanismo, el turismo o la planificación territorial.

2.2. Objetivos específicos

Además del objetivo general, el desarrollo del componente plantea una serie de objetivos específicos que buscan dotar al sistema de funcionalidades concretas y mejorar su utilidad práctica. Estos objetivos se resumen a continuación:

- **Generar una malla 3D del terreno:** Obtener y representar el relieve de una región geográfica mediante una malla tridimensional, a partir de datos abiertos ofrecidos por APIs como *OpenStreetMap*, Mapbox o servicios similares que proporcionen información topográfica o de elevación.
- **Implementar múltiples capas de visualización:** Permitir al usuario seleccionar entre diferentes tipos de capas superpuestas sobre la malla del terreno, tales como vista satelital, vista de calle o una capa personalizada con colores definidos manualmente. Esta funcionalidad facilita distintos modos de análisis y exploración del mapa.
- **Navegación libre por el mapa:** Ofrecer la posibilidad de desplazarse libremente por el mapa tridimensional utilizando controles de navegación intuitivos, además de permitir acciones como hacer *zoom in* y *zoom out* para explorar distintas escalas de visualización del terreno.
- **Funcionalidades personalizables por el usuario:** Incorporar opciones de personalización que permitan al usuario ajustar parámetros como el tamaño del mapa, su posición dentro del entorno 3D, el nivel de detalle o el escalado vertical del relieve. Estas características permiten adaptar la experiencia a distintos casos de uso o preferencias individuales.

2.3. Planificación temporal

La realización de este Trabajo de Fin de Grado se ha desarrollado a lo largo de varios meses, compatibilizándose con las prácticas curriculares en empresa y la asistencia regular a clases presenciales. Esto ha condicionado la disponibilidad horaria, centrando el

grueso del trabajo en fines de semana y periodos no lectivos. A continuación, se detalla la planificación temporal seguida:

- **Septiembre de 2024:** Primer contacto con el tutor del TFG, David Moreno Lumbreras, para explorar posibles temáticas del trabajo.
- **Octubre de 2024:** Reunión con el tutor para concretar la propuesta definitiva, centrada en el desarrollo de un componente para mapas en entornos de realidad virtual, en el marco de la plataforma Babia XR.
- **Octubre - diciembre de 2024:** Fase de documentación y aprendizaje. Se estudian tecnologías clave como *A-Frame* y Babia XR, y se experimenta con componentes ya existentes dentro de esta plataforma. Esta etapa se desarrolló mayoritariamente durante los fines de semana debido a la carga lectiva y laboral (prácticas curriculares y clases los martes y miércoles).
- **Diciembre de 2024:** La universidad facilita el acceso a unas gafas de realidad virtual, permitiendo comenzar el desarrollo y pruebas en entorno real.
- **Febrero de 2025:** Finalizado el periodo de exámenes, se retoma el trabajo de manera más intensiva, comenzando el desarrollo del componente personalizado. En este periodo de tiempo también implementé los conocimientos en *JavaScript* adquiridos en la asignatura de *Aplicaciones Telemáticas* cursada paralelamente
- **Marzo de 2025:** Se alcanza un hito importante: la generación de una malla del terreno y la implementación de una primera capa visual obtenida desde la API de *OpenStreetMap*.
- **Abril de 2025:** Desarrollo de un sistema de capas múltiples, incluyendo la creación de una interfaz gráfica de usuario para gestionar su visualización, así como la incorporación de un mecanismo para trabajar con el token de Mapbox.
- **Mayo de 2025:** Implementación de controles interactivos compatibles con realidad virtual (botones visibles en las gafas VR), así como la posibilidad de modificar coordenadas y niveles de zoom. Redacción y finalización progresiva de la memoria del TFG.

- **Finales de mayo e inicios de junio de 2025:** Corrección y revisión final de la memoria, preparación de la defensa.

Cabe destacar que, aunque las fases descritas se presentan de forma lineal, muchas tareas se han solapado parcialmente. La dedicación al proyecto ha sido principalmente en fines de semana y festivos, dada la simultaneidad con otras actividades académicas y profesionales. Esto también ha significado el no poder tener un plan de reuniones con el tutor, pasando a programar las reuniones (muchas de ellas telemáticamente) cuando podía crearse un hueco libre y el proyecto hubiera avanzado lo suficiente.

A continuación, se muestra un diagrama de Gantt del proceso completo:

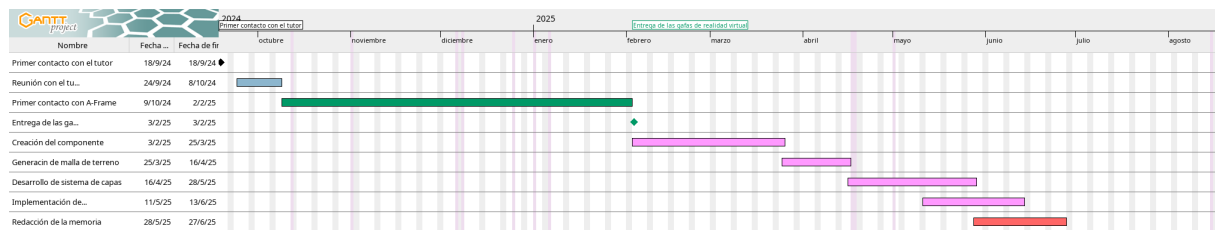


Figura 2.1: Diagrama de Gantt del TFG.

Capítulo 3

Estado del arte

3.1. Babia-XR

Babia XR es una plataforma de desarrollo para entornos inmersivos y experiencias interactivas en realidad virtual (VR), realidad aumentada (AR) y realidad mixta (MR). Se trata de una iniciativa impulsada por investigadores y docentes de la Universidad Rey Juan Carlos (URJC), lo que refuerza su vinculación con el ámbito académico y su aplicación en proyectos de investigación y desarrollo tecnológico.

El objetivo principal de Babia XR es facilitar la creación de aplicaciones inmersivas mediante tecnologías web abiertas, como *WebXR*, *A-Frame* y *Three.js*. Estas tecnologías permiten ejecutar contenidos tridimensionales directamente en navegadores compatibles, sin necesidad de instalar software adicional. Esto representa una ventaja significativa en términos de accesibilidad, portabilidad y compatibilidad con múltiples dispositivos, incluyendo gafas de realidad virtual como las Meta Quest.

En el contexto de este trabajo, Babia XR se utiliza como entorno base para la integración del componente de visualización de mapas tridimensionales. La elección de esta plataforma responde a dos motivos principales:

1. **Necesidad técnica:** proporciona la infraestructura necesaria para desplegar escenas VR complejas, integrando capas de información geoespacial, controles de navegación y representaciones 3D sobre terrenos reales.
2. **Vinculación institucional:** al tratarse de una herramienta desarrollada dentro de

la propia universidad, su uso refuerza la colaboración entre estudiantes y líneas de investigación del centro, además de facilitar el acceso a soporte técnico y documentación específica.

Babia XR se apoya en un conjunto de tecnologías modernas y bien consolidadas:

- *A-Frame*: framework basado en HTML para crear experiencias VR accesibles desde navegadores.
- *Three.js*: biblioteca JavaScript que permite renderizar gráficos 3D en WebGL con gran flexibilidad.
- *WebXR*: estándar para ejecutar experiencias inmersivas directamente en el navegador, compatible con múltiples dispositivos.

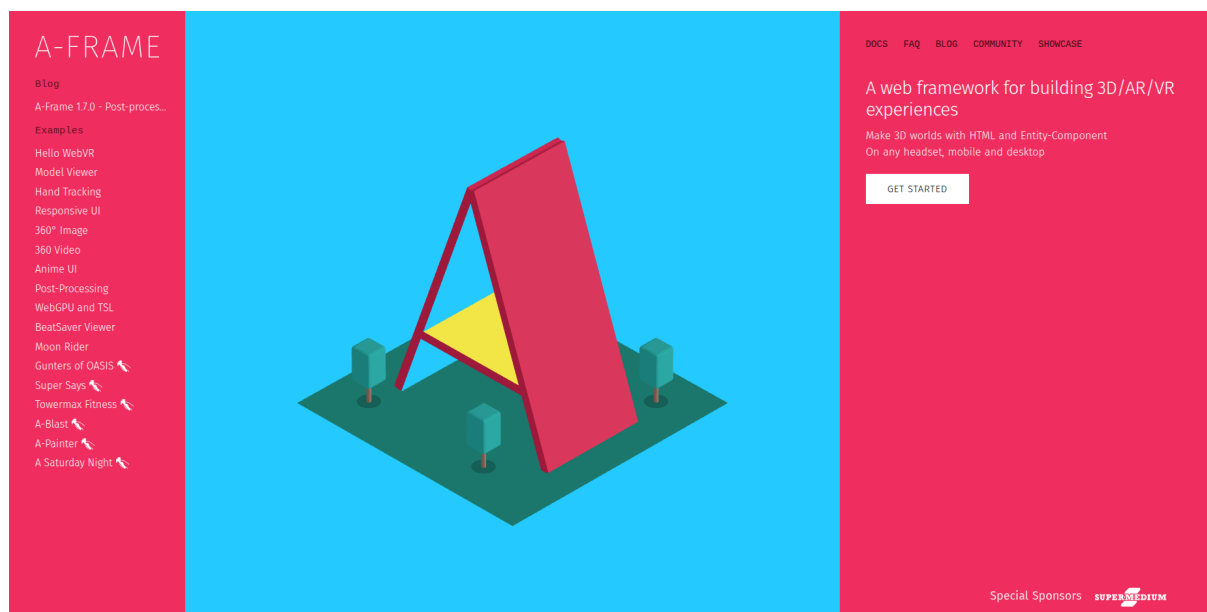
Gracias a esta combinación, Babia XR permite un desarrollo ágil y modular, facilitando la incorporación de componentes personalizados, como el sistema de teselas con desplazamiento que se ha implementado en este proyecto.

3.1.1. A-Frame

A-Frame es un framework de código abierto desarrollado inicialmente por Mozilla para facilitar la creación de experiencias de realidad virtual usando HTML. Su diseño declarativo permite definir escenas tridimensionales mediante etiquetas similares a las del propio lenguaje HTML, lo cual lo hace accesible incluso para desarrolladores con conocimientos limitados de gráficos 3D o WebGL.

A-Frame está construido sobre **Three.js**, lo que le permite mantener una gran potencia gráfica bajo una capa de abstracción más sencilla. Además, es extensible mediante componentes personalizados, lo que permite adaptar el comportamiento de los elementos de la escena a las necesidades específicas del proyecto.

En este trabajo, *A-Frame* actúa como base para representar los elementos visuales del mapa tridimensional, así como para integrar los controles de usuario, cámaras, luces y otros objetos interactivos.

Figura 3.1: Sitio web oficial de *A-Frame*.

3.1.2. Three.js

Three.js es una biblioteca JavaScript muy utilizada para renderizar gráficos tridimensionales en el navegador utilizando **WebGL**. Proporciona una interfaz de alto nivel que simplifica el uso de funcionalidades avanzadas de gráficos 3D, como luces dinámicas, materiales complejos, sombras en tiempo real y animaciones.

En el contexto de Babia XR, Three.js es el motor gráfico que subyace bajo *A-Frame*. Aunque normalmente no se accede directamente a su API cuando se utiliza *A-Frame*, su presencia es fundamental para permitir el renderizado eficiente de los objetos en la escena.

Algunos componentes personalizados del proyecto, especialmente los relacionados con el desplazamiento de texturas y la carga dinámica de teselas, se benefician de las capacidades avanzadas de Three.js, ya que permiten modificar directamente atributos de materiales y geometrías a bajo nivel.

WebGL

WebGL (Web Graphics Library) es una API de JavaScript para renderizar gráficos 2D y 3D dentro de cualquier navegador web compatible sin necesidad de plugins. Está basada en OpenGL ES 2.0 y permite acceder a la aceleración gráfica del hardware de

forma segura. En el contexto del proyecto, WebGL es la tecnología subyacente sobre la que se construyen bibliotecas como Three.js y A-Frame, lo que permite representar escenas tridimensionales complejas directamente en el navegador.

3.1.3. WebXR

WebXR es un estándar del W3C que proporciona una interfaz unificada para acceder a dispositivos de realidad virtual y aumentada desde navegadores web. Permite detectar dispositivos compatibles (como gafas VR o móviles con capacidades AR), obtener sus posiciones y orientaciones en tiempo real, y renderizar escenas de forma inmersiva.

A diferencia de sus predecesores (como WebVR), WebXR está diseñado para soportar tanto realidad virtual como aumentada desde una misma API, y su adopción por parte de los principales navegadores modernos garantiza su viabilidad a largo plazo.

Babia XR utiliza WebXR para habilitar la ejecución de escenas inmersivas en dispositivos como las Meta Quest, permitiendo al usuario entrar en la experiencia VR directamente desde el navegador. Esta capacidad resulta especialmente útil en entornos educativos y de investigación, donde la simplicidad de despliegue es una prioridad.

3.2. HTML

HTML (*HyperText Markup Language*) es el lenguaje estándar para estructurar contenido en la web. En este proyecto, HTML se emplea como base para organizar y enlazar los elementos visuales de la interfaz del componente, además de ser fundamental para la integración de *A-Frame*, que utiliza etiquetas personalizadas dentro de documentos HTML para definir entidades 3D y escenas interactivas.

```

1 <!DOCTYPE html>
2 <html>
3
4 <head>
5   <meta charset="utf-8">
6   <title>A-Frame Babia components</title>
7   <meta name="description" content="Babia Map example" />
8   <script src="https://aframe.io/releases/1.5.0/aframe.min.js"></script>
9   <script src="https://cdn.jsdelivr.net/gh/donmccurdy/aframe-extras@7.2.0/dist/aframe-extras.min.js"></script>
10  <script src="https://unpkg.com/aframe-environment-component@1.3.1/dist/aframe-environment-component.min.js"></script>
11  <script src="https://unpkg.com/aframe-text-geometry-component@0.5.1/dist/aframe-text-geometry-component.min.js"></script>
12  <script src="../../dist/aframe-babia-components.js"></script>
13  <link rel="stylesheet" href="../../assets/gitlab-corner.css">
14 </head>
15
16 <body>
17   <a-scene background="color: #A8F3FF" id="AframeScene">
18     <!-- <a-entity environment></a-entity> -->
19     <a-light type="directional" color="#ffffff" intensity="1.5" position="5 10 5"></a-light>
20
21     <a-assets>
22       <a-asset-item id="optimerBoldFont" src="https://rawgit.com/mrdoob/three.js/dev/examples/fonts/optimer_bold.typeface.json"></a-asset-item>
23     </a-assets>
24
25     <a-entity id="map-terrain" class="collidable" babia-map="zoom: 5; x: 15; y: 12;" position="0 -150 0"></a-entity>
26
27     <a-entity id="right-desktop-ui-position" position="0.25 1.5 -0.5">
28       <!-- Botón para subir de capa -->
29       <a-entity id="zoomInBtn"
30         geometry="primitive: plane; height: 0.075; width: 0.2"
31         material="color: blue"
32         text="value: Zoom In; align: center"
33         position="0 0.05 -0.3"
34         rotation="-30 0 0"
35         class="collidable interactive-button"></a-entity>
36
37       <!-- Botón para bajar de capa -->
38       <a-entity id="zoomOutBtn"
39         geometry="primitive: plane; height: 0.075; width: 0.2"
40         material="color: purple"
41         text="value: Zoom Out; align: center"
42         position="0 -0.05 -0.25"
43         rotation="-30 0 0"
44         class="collidable interactive-button"></a-entity>
45     </a-entity>
46
47     <a-entity id="left-desktop-ui-position" position="-0.25 1.5 -0.5">
48       <!-- Botón para subir de capa -->
49       <a-entity id="zoomInBtn"
50         geometry="primitive: plane; height: 0.075; width: 0.2"
51         material="color: blue"
52         text="value: Zoom In; align: center"
53         position="0 0.05 -0.3"
54         rotation="-30 0 0"
55         class="collidable interactive-button"></a-entity>
56
57       <!-- Botón para bajar de capa -->
58       <a-entity id="zoomOutBtn"
59         geometry="primitive: plane; height: 0.075; width: 0.2"
60         material="color: purple"
61         text="value: Zoom Out; align: center"
62         position="0 -0.05 -0.25"
63         rotation="-30 0 0"
64         class="collidable interactive-button"></a-entity>
65     </a-entity>
66   </a-scene>
67 </body>
68 </html>

```

Figura 3.2: Código HTML del componente.

3.3. JavaScript y Node.js

JavaScript es el lenguaje de programación principal del entorno web, y se utiliza tanto en el desarrollo de interfaces interactivas como en la lógica del componente desarrollado. Permite manipular elementos HTML, responder a eventos del usuario y controlar el comportamiento del componente en tiempo real.

```

aframe-babia-components > components > visualizers > js babia-map.js >  init
182 buildMesh: function (data, el) {
248   const fragmentShader =
249   ";
250   // CREAR TEXTURA DESDE IMAGEN
251   async function loadDisplacementMap(url) {
252     let img = new Image();
253     img.src = url;
254     img.crossOrigin = "Anonymous"; // Evitar problemas CORS
255
256     return new Promise((resolve, reject) => {
257       img.onload = () => {
258         const canvas = document.createElement("canvas");
259         canvas.width = img.width;
260         canvas.height = img.height;
261         const ctx = canvas.getContext("2d");
262
263         // Dibujar la imagen en el canvas de forma invertida (la orientación de la imagen es diferente en canvas respecto a WebGL)
264         ctx.translate(0, canvas.height);
265         ctx.scale(1, -1);
266         ctx.drawImage(img, 0, 0);
267         const imageData = ctx.getImageData(0, 0, img.width, img.height);
268         const data = imageData.data;
269
270         // Crear un array para la textura de desplazamiento
271         const displacement = new Float32Array(img.width * img.height);
272
273         let elevation;
274         let auxMinHeight = 1;
275         let auxMaxHeight = 0;
276
277         for (let i = 0; i < data.length; i += 4) {
278           const R = data[i];
279           const G = data[i + 1];
280           const B = data[i + 2];
281
282           // Fórmula de Mapbox terrain-RGB -> Elevación en metros
283           elevation = (R * 256 * 256 + G * 256 + B) * 0.1 - 10000;
284
285           // Normalizar para Three.js (opcional, si necesitas valores entre 0 y 1)
286           elevation = (elevation + 10000) / 20000; // Elevación entre 0 y 1
287
288           // Comprobar el valor más bajo y más alto
289           if (elevation < auxMinHeight) {
290             auxMinHeight = elevation;
291           }
292           if (elevation > auxMaxHeight) {
293             auxMaxHeight = elevation;
294           }

```

Figura 3.3: Código fuente en *JavaScript* del componente.

3.3.1. Node.js y herramientas asociadas

Node.js es un entorno de ejecución para JavaScript fuera del navegador. Aunque el componente se ejecuta en el cliente, Node.js se utiliza durante el desarrollo para gestionar dependencias, ejecutar scripts y automatizar tareas. Para ello, se hace uso de **npm** (Node Package Manager), el gestor de paquetes de Node.js, y de **nvm** (Node Version Manager), que facilita la instalación y gestión de múltiples versiones de Node.js en un mismo sistema. Estas herramientas son necesarias para compilar, organizar y desplegar el componente de manera eficiente.

3.4. APIs de Mapbox y *OpenStreetMap*

Mapbox y *OpenStreetMap* son servicios que proporcionan datos cartográficos. *OpenStreetMap* (OSM) es una iniciativa colaborativa que ofrece mapas del mundo creados por

una comunidad de voluntarios. Por su parte, Mapbox utiliza datos de OSM y otras fuentes para ofrecer un conjunto de APIs avanzadas orientadas al desarrollo web, como mapas personalizados, capas de estilo y servicios de geolocalización.

En este proyecto se hace uso de estas APIs para obtener datos de elevación y estilos visuales (como capas satelitales o vistas de calle) que se aplican sobre la malla del terreno generada en el componente. Mapbox requiere un *token* de acceso personal, mientras que OSM puede utilizarse directamente mediante peticiones a sus servidores abiertos.

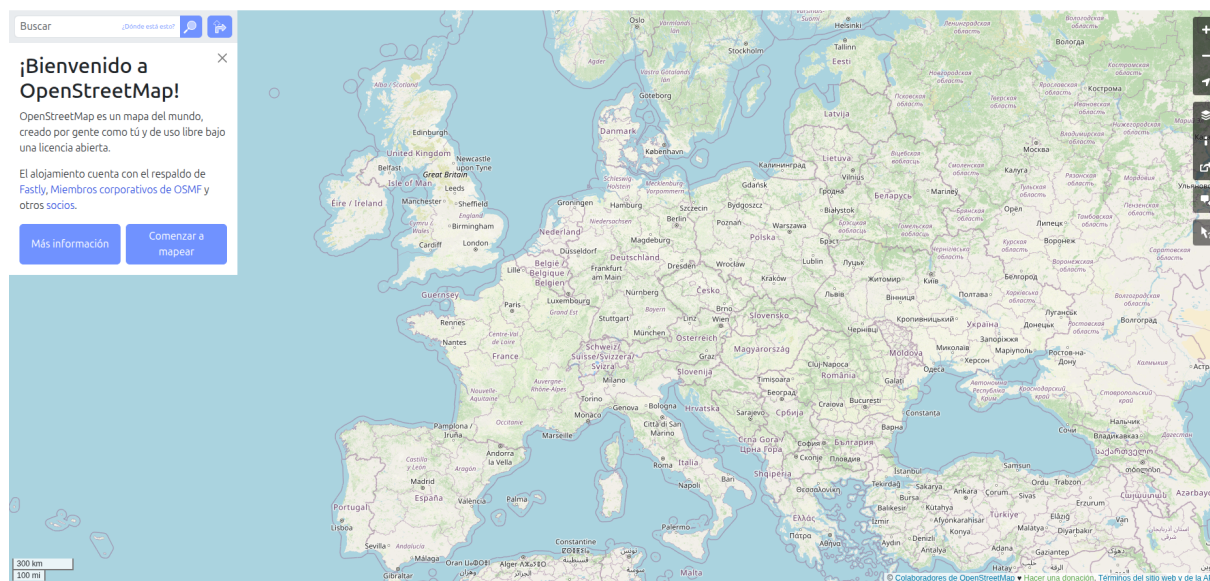


Figura 3.4: Sitio web oficial de *OpenStreetMap*.

3.5. Gafas de Realidad Virtual: Meta Quest 3

Las Meta Quest 3, desarrolladas por Meta (anteriormente Oculus), son unas gafas de realidad virtual autónomas que permiten ejecutar aplicaciones VR sin necesidad de un ordenador externo. Disponen de controladores hápticos, sensores de movimiento y capacidad de realidad mixta (passthrough a color), lo que las convierte en una herramienta ideal para pruebas e implementación de experiencias inmersivas.

Gracias al soporte de WebXR en su navegador interno, estas gafas permiten ejecutar el componente desarrollado directamente desde la web, lo que facilita el proceso de validación en un entorno real.



Figura 3.5: Gafas prestadas por la universidad.

3.6. Editor de Código: Visual Studio Code

Visual Studio Code (VS Code) es un editor de código fuente desarrollado por Microsoft, ampliamente adoptado por la comunidad de desarrolladores. Ofrece integración con sistemas de control de versiones, depuración, resaltado de sintaxis y soporte para extensiones que facilitan el desarrollo web y VR.

Durante el desarrollo del proyecto, se ha utilizado como entorno principal de trabajo, aprovechando su integración con herramientas como Git, su terminal integrada para la ejecución de comandos Node.js y su compatibilidad con tecnologías web modernas.

3.7. Apoyo de herramientas de inteligencia artificial

Durante el desarrollo del proyecto, se ha contado con el apoyo de herramientas basadas en inteligencia artificial como ChatGPT, que han sido útiles para resolver dudas técnicas,

refinar la redacción de la documentación, generar fragmentos de código y comprender tecnologías específicas de forma más ágil.

Estas herramientas, si bien no sustituyen el conocimiento técnico, han servido como soporte complementario para acelerar el proceso de desarrollo, ahorrar tiempo en tareas repetitivas y mejorar la calidad de la documentación generada.

Capítulo 4

Diseño e implementación

Durante el mes de septiembre de 2024 inicié el contacto con mi tutor de Trabajo de Fin de Grado con el objetivo de plantear una propuesta de proyecto basada en la tecnología A-Frame. La elección de esta tecnología estuvo motivada por varios factores. En primer lugar, ya existía una base previa de desarrollo gracias al proyecto *BabiaXR*, realizado por personal docente de la Universidad, lo cual me permitiría apoyarme en ejemplos funcionales y resolver dudas de forma más ágil. En segundo lugar, A-Frame ofrecía un campo aún por explorar en profundidad en distintos aspectos de la realidad extendida, lo que abría la posibilidad de aportar una solución original y útil.

Finalmente, me decanté por diseñar un visor de mapas tridimensionales para realidad extendida, compatible tanto con realidad virtual como con realidad aumentada. La representación de mapas en entornos de realidad virtual presenta diversos desafíos: muchos de los visores existentes son complejos de manejar, poco interactivos o requieren una gran cantidad de recursos computacionales. El objetivo de este trabajo ha sido desarrollar un componente personalizado, llamado **babia-map**, que permita representar mapas en 3D de forma eficiente y accesible en A-Frame, optimizando tanto la experiencia de usuario como el rendimiento gráfico.

Para lograr este objetivo, fue imprescindible adquirir un conocimiento detallado sobre el funcionamiento interno de A-Frame. Esta herramienta está construida sobre la librería gráfica `three.js`, que ofrece una capa de abstracción para desarrollar experiencias WebXR utilizando HTML y componentes declarativos. A lo largo del desarrollo, fue necesario aprender a manipular directamente tanto entidades de A-Frame como estructuras nativas

de `three.js`, incluyendo geometrías, materiales, texturas y sombreadores (*shaders*).

El código fuente se debía realizar en JavaScript, pues A-Frame funciona dentro de HTML. *NodeJS* se puede definir como el *software* para JavaScript equivalente a lo que es *Django* para *Python*. Se utilizaron varios comandos cada vez que se accedía al entorno de desarrollo. Primeramente se realizó un fork del proyecto original y después se clonó el repositorio en local.

Para utilizar *NodeJS* se necesita el comando `nvm`. Con el comando `nvm use stable` se establecería la versión actual de *NodeJS* en el directorio de trabajo. En mi caso, he estado trabajando con una versión más antigua, así que he utilizado el comando `nvm use 21.7.1`. Es necesario ejecutar este comando por cada terminal nueva que se despliegue, pues la versión configurada por defecto no tiene por qué ser esta. Para comprobar la versión, es necesario utilizar el comando `nvm ls`.

Nada más clonar el repositorio, fue imprescindible realizar un `npm install` en el directorio del repositorio, lo que instaló todas las dependencias especificadas en el fichero llamado `afreame-babia-components/package.json`. Se puede definir el comando `npm` como el equivalente en JavaScript de `pip` en Python.

Es muy importante ejecutar el comando `npm run dist` cada vez que se modifique un componente para poder actualizar el directorio de distribuciones. Para trabajar en el servidor, el comando para lanzarlo es `npm run ssldev`.

4.1. Proceso de formación

Desde octubre de 2024 hasta enero de 2025, dediqué una fase inicial a experimentar con componentes existentes del ecosistema BabiaXR, lo cual me permitió familiarizarme con la lógica de desarrollo modular de A-Frame. Esta fase incluyó pruebas en local, modificaciones de componentes ya existentes y la exploración de nuevas funcionalidades.

En diciembre de 2024 recibí unas gafas Meta Quest 3, lo que marcó un punto de inflexión en el proyecto: a partir de entonces pude visualizar los resultados del desarrollo directamente en un entorno de realidad virtual, lo cual facilitó significativamente la detección de errores de usabilidad, escalado, perspectiva o rendimiento.

Uno de los primeros pasos técnicos fue realizar una revisión de componentes similares

ya disponibles. Entre ellos, encontré **aterrain**, un componente para generar terreno 3D en A-Frame basado en mapas de altura (*heightmaps*). Sin embargo, este componente estaba obsoleto, sin mantenimiento activo desde hacía varios años, y resultaba difícil de modificar o ampliar. Esta limitación me llevó a desarrollar una solución propia desde cero, basada en un enfoque más flexible y adaptado a los requerimientos actuales de WebXR.

En paralelo, comencé a investigar cómo obtener y renderizar teselas virtuales (*tiles*) que sirvieran como base para construir el mapa tridimensional. Esta búsqueda me llevó a estudiar el uso de mapas de altura codificados en imágenes RGB, así como técnicas de sombreado utilizando **WebGL** para representar distintas elevaciones y biomas de forma visual.

Gran parte del conocimiento adquirido provino de la consulta de recursos disponibles en línea, especialmente vídeos tutoriales que explicaban el proceso de generación de mallas tridimensionales a partir de mapas de altura, así como su posterior coloración mediante sombreadores. Estos recursos fueron esenciales para entender cómo convertir una imagen en una geometría modificable y cómo aplicar efectos visuales mediante programación gráfica.

A continuación, muestro las tareas que se propusieron para esta primera fase del proyecto:

Tareas a realizar

■ Datos

- Conseguir datos geolocalizados (*GeoJSON*).
- Transformar datos a listas de objetos *JSON*.
- Buscar servicios de teselas virtuales.

■ Exploratorio

- Buscar componentes *A-Frame* que muestren mapas (seguramente ya haya cosas hechas en GitHub).
- Bibliotecas JavaScript para crear mapas (por ejemplo, *OpenStreetMap*). Esto se refleja en el apartado 3 de Estado del Arte (cosas que ya se han hecho).

■ Desarrollo

- Desarrollar el componente *babia-map*. En este caso específico se utilizó el fichero `aframe-babia-components/components/visualizers/babia-map.js`, especificado en `aframe-babia-components/index.js`. Para integrar todo en la página local, se debió añadir en `aframe-babia-components/index.html`.
- Se necesitará utilizar:
 - Babia XR.
 - A-Frame Components.
 - Bibliotecas JS/HTML/A-Frame.

Luego, las tareas acabaron en el siguiente estado:

Tareas realizadas

GeoJSON

Se revisaron los fundamentos del formato *GeoJSON*, una especificación en JavaScript que permite representar distintos tipos de geometrías (puntos, líneas, polígonos, multipuntos, multilíneas y multipolígonos). Cada uno de estos elementos se definía mediante tres atributos principales: tipo, geometría y propiedades. Además, se tuvo en cuenta el uso del atributo *Bounding Box*, empleado para delimitar el espacio de un objeto y agilizar su consulta. La referencia principal fue el RFC 7946 (<https://datatracker.ietf.org/doc/html/rfc7946>), donde también se establecía que el sistema de coordenadas utilizado era WGS 84.

Componentes *A-Frame* para mapas

Durante el desarrollo, se exploró el uso del componente *a-terrain*, actualizado por última vez hace seis años. Para su integración en el repositorio del proyecto Babia, se creó el fichero `dist/aframe-aterrain-components.js` y se añadió un directorio `js` con el archivo `examples/js/aframe-orbit-controls-component.js`. También se configuró una vista de pruebas en la página principal dentro del directorio `aframe-babia-components/examples/demos/te` con el objetivo de visualizar los resultados del componente.

Se siguieron las instrucciones de instalación recogidas en la documentación del componente (<https://github.com/anselm/aterrain/tree/master>), aunque se detectaron

incompatibilidades con versiones recientes de la librería *three*, lo que generó dudas sobre la funcionalidad total del componente en entornos actuales.

Finalmente, tras comprender que el esfuerzo de integrar cambios en un código plenamente desarrollado podía ser mayor que utilizar las herramientas y versiones modernas de A-Frame, se descartó usar este componente externo a BabiaXR.

Librería de JavaScript para mapas

Se estudió el uso de *OpenStreetMap* (OSM) en conjunto con la librería *Leaflet*, una herramienta de código abierto ampliamente utilizada para representar mapas interactivos en la web. La documentación consultada fue la disponible en su página oficial (<https://leafletjs.com/reference.html>).

Leaflet es una librería muy precisa y útil para desarrollar en JavaScript; sin embargo, no sería de utilidad para componentes en tres dimensiones. Fue entonces cuando se comprendió la necesidad de encontrar una solución que trabajara directamente con OSM.

Teselas virtuales

Se investigó la plataforma MapTiler (<https://docs.maptiler.com/sdk-js/examples/3d-map/>), que ofrecía APIs para obtener teselas con datos geoespaciales. Se creó una cuenta personal y se generó una clave de acceso para la API. Asimismo, se instaló el paquete correspondiente mediante `npm install -save @maptiler/sdk`, como paso previo a la integración con el proyecto.

Un *SDK* (*Software Development Kit*, o kit de desarrollo de software) es un conjunto de herramientas, bibliotecas, documentación y ejemplos que permiten a los desarrolladores integrar funcionalidades específicas en sus propias aplicaciones de forma más sencilla y eficiente. En este proyecto se planteó la utilización de *MapTiler SDK for JavaScript*, una biblioteca que proporciona interfaces para incorporar mapas interactivos, capas geoespaciales y servicios de teselas en aplicaciones web.

El uso de un SDK como el de MapTiler facilita tareas complejas como la gestión de teselas, las proyecciones geográficas, la interacción con el usuario o la carga dinámica de datos cartográficos. Gracias a esta abstracción, el desarrollo se vuelve más ágil y centrado en la lógica propia del proyecto, sin necesidad de implementar desde cero un motor de visualización geoespacial.

No obstante, esta solución solo funciona para un desarrollador. Para utilizar algo visible en la web para cualquier usuario, era necesario la utilización de sus diversas APIs.

Uso de *Three.js* y mapas de altura

Se revisó el funcionamiento de *Three.js*, la librería sobre la que se construye *A-Frame*, para representar terrenos tridimensionales mediante el uso de mapas de altura. Estos mapas consistían en imágenes en blanco y negro donde el valor de gris indicaba la altitud relativa.

Se experimentó con propiedades como `displacementMap` o `bumpTexture` para referenciar la imagen del mapa, y `displacementScale` o `bumpScale` para controlar la escala del relieve. Además, se utilizaron los atributos `width` y `height` para definir las dimensiones del plano.

Para representar distintos biomas según la elevación, se aplicaron *shaders* personalizados, adaptados a partir de varios tutoriales encontrados en YouTube: *How to quickly make terrain using a heightmap three.js* (<https://www.youtube.com/watch?v=wULUAhckH9w>) y *How to color terrain from heightmap three.js* (<https://www.youtube.com/watch?v=G0hWjD0n46c>). Estos *shaders* permitían colorear automáticamente el terreno con zonas de agua, vegetación, roca o nieve, basándose en el valor de altura normalizado entre 0.0 y 1.0.

Acceso a imágenes RGB desde la API de MapBox

Finalmente, se revisaron los formatos de acceso a datos cartográficos. Se distinguió entre los formatos de teselas de OpenStreetMap (<https://tile.openstreetmap.org/zoom/x/y.png>) y los de elevación disponibles a través de Mapbox, que permiten obtener imágenes RGB donde la codificación del color representa la altura.

La documentación de referencia fue <https://docs.mapbox.com/api/maps/raster-tiles/>, y se consultó un ejemplo práctico desde <https://docs.mapbox.com/data/tilesets/guides/access-elevation-data/>. Se probó el uso del parámetro opcional `@2x`, que permite solicitar imágenes de mayor resolución (512x512) en lugar de las predeterminadas (256x256), con el objetivo de reducir el número total de solicitudes al cubrir un área más amplia por imagen.

4.2. Arquitectura general

Una vez terminada la fase de formación, comencé a desarrollar el componente como tal. Aprovechando la capacidad de *Three.js* de crear mallas capaces de simular un terreno gracias a una imagen en blanco en negro, se utilizó la API de MapBox para obtener una imagen de un mapa en formato RGB. Lo que faltaba era convertir esa imagen a escala de grises para que *Three.js* por sí solo fuese capaz de interpretar la información.

Durante esta fase del desarrollo, el componente **babiaxr** fue diseñado con una serie de propiedades configurables que permiten ajustar tanto la geometría del terreno como las texturas empleadas para su visualización en tiempo real. A continuación se describen las propiedades disponibles:

Cuadro 4.1: Propiedades definidas en el componente **babiaxr**

Propiedad	Tipo	Valor por defecto	Descripción
<code>width</code>	number	1000	Anchura total del plano que representa el terreno, en unidades del sistema de coordenadas de A-Frame.
<code>height</code>	number	1000	Altura total del plano del terreno.
<code>horTexture</code>	number	2	Número de teselas horizontales que se cargarán y dispondrán en el terreno.
<code>vertTexture</code>	number	2	Número de teselas verticales. En combinación con <code>horTexture</code> , define una malla de $n \times m$ imágenes.
<code>widthSegments</code>	int	200	Número de subdivisiones horizontales del plano, importante para un desplazamiento de vértices más suave.
<code>heightSegments</code>	int	200	Subdivisiones verticales del plano. Cuantos más segmentos, mayor detalle en el relieve.

Continúa en la siguiente página

Cuadro 4.1 – Continuación de la página anterior

Propiedad	Tipo	Valor por defecto	Descripción
<code>bumpScale</code>	number	300	Factor de escala para la elevación en el mapa de alturas. Define cuánto sobresale el relieve respecto al plano original.
<code>wireframe</code>	boolean	false	Permite renderizar el terreno como una malla en forma de alambre (wireframe), útil para depuración o estilo técnico.
<code>token</code>	string	—	Clave de acceso (API Token) necesaria para servicios que lo requieren, como Mapbox.
<code>subdomain</code>	string	“a”	Subdominio utilizado en servicios como Thunderforest, que distribuyen la carga en subdominios (a, b, c).
<code>textureType</code>	string	“Standard”	Tipo de textura a utilizar: Standard , Satellite , Hot , Cycle o Transport . Cada uno corresponde a una API distinta.

Estas propiedades permiten una configuración flexible del componente, adaptándose a diferentes resoluciones, estilos visuales y requerimientos técnicos. En particular, el uso de

La función `loadDisplacementMap` tiene como objetivo generar una textura de desplazamiento en formato `THREE.DataTexture`, a partir de una imagen RGB codificada según el estándar *Mapbox Terrain-RGB*, el cual representa la elevación del terreno mediante valores en los canales rojo, verde y azul. La figura 4.1 muestra un diagrama de flujo del funcionamiento de esta parte del código.

El proceso comienza con la creación de un objeto **Image**, al cual se le asigna la ruta del recurso (`url`) y se habilita el atributo `crossOrigin` con el valor `“anonymous”` para evitar errores relacionados con el uso compartido de recursos entre distintos orígenes (CORS).

Una vez que la imagen se ha cargado correctamente, se dibuja sobre un objeto **canvas** de HTML, invirtiendo previamente su eje vertical para corregir la orientación con respecto a WebGL. A continuación, se accede a los datos de píxeles mediante el contexto 2D del

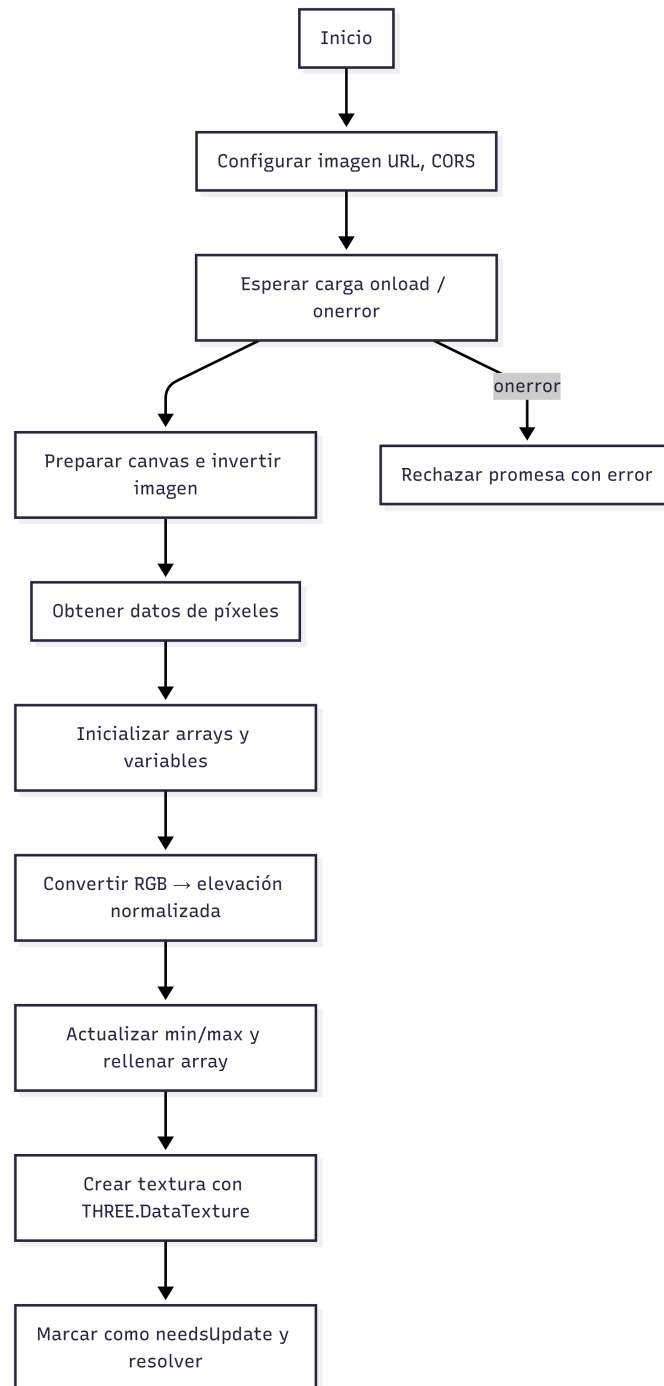


Figura 4.1: Función para generar imagen en escala de grises

canvas y se extrae un array de valores RGBA.

El núcleo del algoritmo reside en la iteración sobre los valores RGB de cada píxel. Utilizando la fórmula proporcionada por Mapbox, se calcula la elevación en metros:

$$\text{elevación} = (R \cdot 256^2 + G \cdot 256 + B) \cdot 0,1 - 10,000$$

Posteriormente, se normaliza este valor al rango $[0, 1]$ para adaptarlo al sistema de representación de Three.js, dividiendo por el intervalo total de 20 000 metros (de $-10,000$ a $+10,000$). Durante la iteración, también se registran los valores mínimos y máximos de elevación con el fin de facilitar ajustes posteriores al renderizado o a la aplicación de sombreados basados en altura.

Una vez procesados todos los píxeles, se construye una textura de tipo `THREE.DataTexture` utilizando los valores de elevación normalizados. Esta textura está compuesta por un único canal (`THREE.RedFormat`) y almacena valores en coma flotante (`THREE.FloatType`), lo que permite una representación precisa de las alturas en aplicaciones tridimensionales.

Finalmente, se marca la textura como lista para su uso (`texture.needsUpdate = true`) y se resuelve la promesa con el objeto de textura generado. En caso de que ocurra un error durante la carga de la imagen, se rechaza la promesa de forma adecuada.

Definiciones y conceptos clave

- **Canvas:** En el contexto de la programación web, un *canvas* es un elemento HTML que proporciona un área rectangular donde se puede dibujar gráficos mediante *scripting*, típicamente usando JavaScript. Este elemento permite renderizar imágenes, gráficos, animaciones y manipular píxeles de forma dinámica. En el código analizado, el canvas se utiliza para procesar y transformar la imagen descargada, facilitando la extracción de datos de píxeles necesarios para generar el mapa de desplazamiento.
- **Promesa (*Promise*):** Una promesa es un objeto en JavaScript que representa la eventual finalización (o fallo) de una operación asíncrona y su valor resultante. Las promesas permiten manejar tareas que tardan un tiempo en completarse, como la carga de imágenes o datos desde la red, evitando el bloqueo del hilo principal de ejecución. En este código, la función `loadDisplacementMap` devuelve una promesa que se resuelve con la textura cargada o se rechaza en caso de error.

- **Propiedad `crossOrigin`:** Esta propiedad de un objeto `Image` en JavaScript especifica si la imagen debe ser cargada utilizando credenciales para solicitudes de origen cruzado (*cross-origin*). El valor `."anonymous"` indica que la imagen se solicita sin enviar *cookies*, credenciales HTTP o certificados TLS, lo que es necesario para evitar problemas de seguridad relacionados con la política del mismo origen (*Same-Origin Policy*). Esto permite que la imagen pueda ser utilizada en un *canvas* para extraer datos de píxeles sin que el navegador bloquee esta operación debido a restricciones de seguridad. En resumen, definir `crossOrigin = ."anonymous"` es crucial para poder manipular imágenes procedentes de dominios diferentes al del sitio web sin incurrir en errores de seguridad o *tainting* del canvas.

Una vez se ha cargado la imagen, se crea la malla y se le añade como textura el canvas generado.

El código presentado tiene como objetivo principal la construcción de una malla tridimensional que representa un terreno, utilizando mapas de desplazamiento y diferentes tipos de texturas procedentes de diversas fuentes, como Mapbox o OpenStreetMap. La figura 4.2 muestra un diagrama de flujo del funcionamiento de esta parte del código. A continuación se describe en detalle el funcionamiento del mismo.

En primer lugar, se construye una URL para obtener un mapa de desplazamiento desde la API de Mapbox, empleando parámetros dinámicos como el nivel de zoom, las coordenadas de teselas (x, y) y un token de acceso. Esta URL se utiliza para cargar una textura que contiene información sobre la elevación del terreno en formato RGB.

A continuación, se invoca la función `loadDisplacementMap` con dicha URL. Esta función devuelve una promesa que, cuando se resuelve correctamente, proporciona la textura con la información de desplazamiento. Si la carga falla, se captura el error y se informa mediante un mensaje en la consola.

Cuando la promesa se resuelve, se procede a configurar las propiedades de repetición horizontal y vertical de la textura para permitir que esta se repita en ambas direcciones según los parámetros `horTexture` y `vertTexture` definidos en `data`. Esto es importante para evitar que la textura se estire o se muestre de forma incorrecta en la geometría final.

Seguidamente, se definen dos variables fundamentales: `uniforms` y `groundMaterial`. La variable `uniforms` contendrá los valores y texturas que serán enviados a los shaders

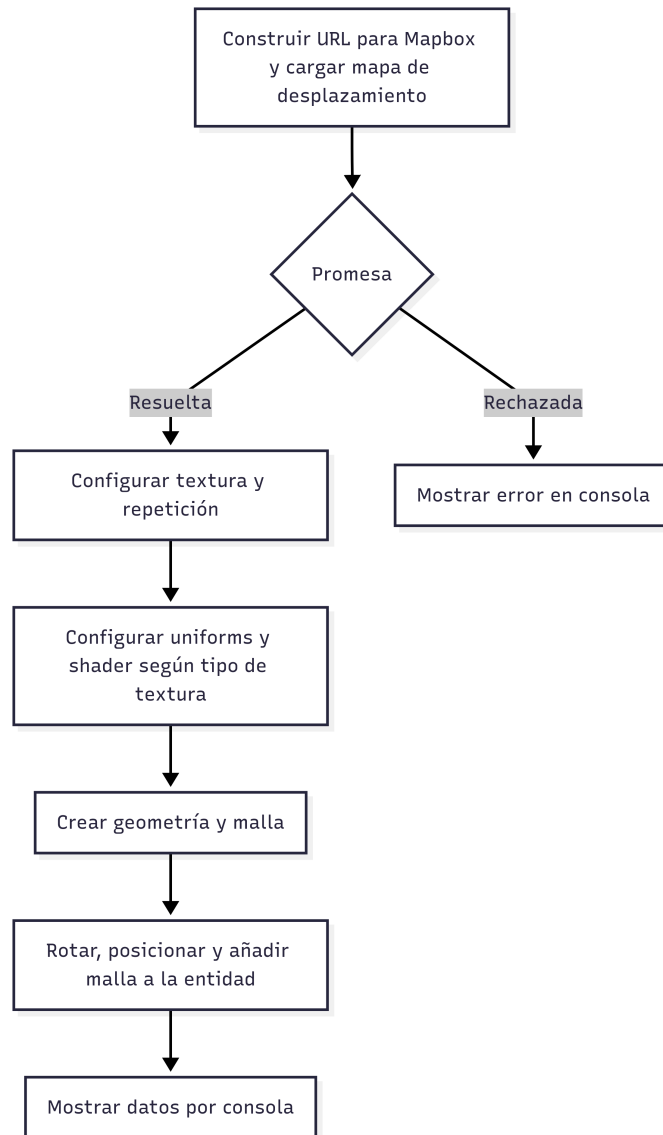


Figura 4.2: Creación de la malla mediante una función flecha

para la renderización, mientras que `groundMaterial` será el material shader aplicado a la malla.

El código contempla dos ramas principales dependiendo del tipo de textura especificado en `data.textureType`:

- Si el tipo es `Coloured`, se configuran los *uniforms* con la textura de desplazamiento, la escala del bump mapping, y los valores mínimos y máximos de elevación obtenidos previamente. Posteriormente, se crea un `ShaderMaterial` que utiliza shaders personalizados (`vertexShader1` y `fragmentShader1`) para renderizar la malla con el efecto deseado.
- Para otros tipos de textura, el código selecciona una URL distinta según el tipo indicado (por ejemplo, `Standard`, `Satellite`, `Hot`, `Cycle` o `Transport`). Se carga la textura correspondiente mediante `THREE.TextureLoader`. Después, se configuran los *uniforms* para incluir tanto la textura de desplazamiento como la textura cargada, y se crea un `ShaderMaterial` con shaders alternativos (`vertexShader2` y `fragmentShader2`).

El componente utiliza diferentes proveedores de mapas a través de sus APIs para renderizar imágenes de teselas (*tiles*) en función del tipo de textura seleccionado. Cada API devuelve una imagen rasterizada (formato `.png`) correspondiente a una porción del mapa en coordenadas `x`, `y` y un nivel de `zoom` determinado.

- **OpenStreetMap Estándar (Standard):**

`https://tile.openstreetmap.org/{zoom}/{x}/{y}.png`

Este servicio proporciona el mapa base clásico de OpenStreetMap. Es de uso libre bajo la licencia ODbL (*Open Database License*), siempre que se acredite debidamente la fuente (© *OpenStreetMap contributors*).

- **Mapbox Satélite (Satellite):**

`https://api.mapbox.com/styles/v1/mapbox/satellite-v9/tiles/256/{zoom}/{x}/{y}?access_token={token}`

Mapbox ofrece imágenes satelitales de alta resolución. Su uso requiere el registro en su página (`https://mapbox.com`) para obtener un token de acceso (API Key). El

servicio tiene un plan gratuito limitado, por lo que es necesario controlar el número de solicitudes mensuales.

- **Humanitarian OpenStreetMap Team (Hot):**

`https://tile-a.openstreetmap.fr/hot/{zoom}/{x}/{y}.png`

Esta variante cartográfica está pensada para entornos humanitarios. Se basa en datos de OSM con un estilo visual que resalta elementos clave como edificaciones, caminos, zonas de riesgo y refugios. Es de uso libre, pero conviene no abusar del servicio si no se tiene un acuerdo explícito.

- **OpenCycleMap (Cycle):**

`https://{subdomain}.tile.thunderforest.com/cycle/{zoom}/{x}/{y}.png`

Proporcionado por *Thunderforest*, está enfocado a ciclistas. Muestra carriles bici, desniveles, rutas populares y puntos de interés. Aunque su acceso es libre mediante sus dominios públicos, Thunderforest requiere el uso de una clave de API desde junio de 2020. Se puede obtener registrándose en su sitio web: <https://www.thunderforest.com/docs/apikeys/>.

- **Thunderforest Transport:**

`https://{subdomain}.tile.thunderforest.com/transport/{zoom}/{x}/{y}.png`

También gestionado por Thunderforest, este estilo de mapa está centrado en el transporte público. Resalta líneas de autobús, tren, metro, estaciones y rutas frecuentes. Igual que el mapa ciclista, su uso requiere una clave de API gratuita o de pago.

A continuación, se muestra un resumen comparativo de las texturas:

Nombre	Proveedor	Requiere API Key	Tipo de mapa
Standard	OpenStreetMap	No	Topográfico general
Satellite	Mapbox	Sí	Imagen satelital real
Hot	OSM France (HOT)	No	Estilo humanitario
Cycle	Thunderforest	Sí	Enfocado en ciclismo
Transport	Thunderforest	Sí	Transporte público

Cuadro 4.2: Comparación de los estilos de mapa disponibles

Cada uno de estos mapas ofrece una visión distinta del terreno y puede utilizarse en función del contexto de la aplicación: por ejemplo, el estilo *Satellite* es ideal para aplicaciones inmersivas realistas, mientras que *Cycle* o *Transport* se prestan mejor a interfaces informativas.

Una vez configurado el material, se genera la geometría del terreno mediante un plano (`PlaneGeometry`) con las dimensiones y subdivisiones indicadas en `data`. Esta geometría representa la superficie sobre la que se aplicará la textura y el desplazamiento.

A partir de la geometría y el material definidos, se crea la malla 3D (`Mesh`) que representa el terreno. Se ajusta su rotación para que quede orientada correctamente (girada 90 grados sobre el eje X) y se posiciona en el espacio 3D.

Finalmente, esta malla se añade a la entidad correspondiente en la escena 3D mediante el método `setObject3D`, permitiendo que forme parte del renderizado. El estado y los datos actuales se imprimen en la consola para facilitar el seguimiento y depuración del proceso.

En caso de error durante la carga del mapa de desplazamiento, se captura el fallo y se muestra un mensaje en la consola para alertar al usuario o desarrollador.

En resumen, este código permite cargar y preparar una textura de desplazamiento a partir de datos de elevación, aplicar distintos tipos de texturas visuales sobre una geometría plana subdividida, y finalmente crear una malla con shaders personalizados que simulan un terreno 3D realista en una escena WebGL gestionada con la biblioteca `three.js`.

WebGL (Web Graphics Library) es una API de JavaScript para renderizar gráficos 3D y 2D interactivos dentro de cualquier navegador compatible, sin necesidad de instalar

plugins adicionales. Se basa en *OpenGL ES*, una versión reducida de OpenGL diseñada para sistemas embebidos como móviles o navegadores.

WebGL permite que los desarrolladores creen visualizaciones gráficas complejas, simulaciones físicas, juegos 3D o aplicaciones de realidad virtual directamente en la web. Gracias a su integración con HTML5, WebGL es una tecnología fundamental para interfaces inmersivas y entornos tridimensionales. ¿Por qué es importante?

- **Multiplataforma:** Funciona en navegadores modernos tanto en ordenadores como en dispositivos móviles.
- **Basado en GPU:** Aprovecha el hardware gráfico para obtener un alto rendimiento en el renderizado.
- **Estándar abierto:** Permite trabajar con bibliotecas como `Three.js`, `Babylon.js` o frameworks XR como `A-Frame`.
- **Clave en visualización interactiva:** Especialmente útil en realidad virtual, videojuegos, visualización científica o mapas 3D.

A continuación, se describe en lenguaje natural el funcionamiento aproximado de los shaders personalizados utilizados en el componente.

Vertex Shader 1: deformación del terreno por textura de relieve

Entrada:

- `bumpTexture`: imagen donde la intensidad del canal rojo representa la altura
- `bumpScale`: escala de altura
- `position`: posición original del vértice
- `normal`: vector normal del vértice

Pasos:

1. Leer la altura desde la textura `bumpTexture` usando las coordenadas UV.
2. Guardar la altura en `vAmount` (usada luego por el fragment shader).

3. Desplazar el vértice en la dirección de su normal usando la altura y la escala.
4. Calcular la posición final del vértice transformado para renderizarlo en pantalla.

Fragment Shader 1: colores según elevación

Entrada:

- vAmount: altura del vértice
- minHeight, maxHeight: límites del rango de alturas

Pasos:

1. Calcular el rango total de alturas.
2. Definir umbrales para distintos biomas: agua, arena, hierba, roca y nieve.
3. Aplicar funciones smoothstep para generar transiciones suaves entre biomas.
4. Asignar un color según el bioma en el que se encuentra la altura.
5. Combinar los colores y establecer el color final del píxel.

Vertex Shader 2: deformación y paso de coordenadas UV

- Igual al vertexShader1, pero además guarda las coordenadas UV en vUv para que puedan usarse en el fragment shader.

Fragment Shader 2: aplicación de textura RGB

Entrada:

- mapTexture: textura de colores reales (como una imagen satelital)
- vUv: coordenadas UV del fragmento actual

Pasos:

1. Obtener el color del píxel desde la textura.
2. Dibujar ese color en pantalla (gl_FragColor).

Capítulo 5

Experimentos y validación

Este capítulo se introdujo como requisito en 2019. Describe los experimentos y casos de test que tuviste que implementar para validar tus resultados. Incluye también los resultados de validación que permiten afirmar que tus resultados son correctos.

Capítulo 6

Resultados

En este capítulo se incluyen los resultados de tu trabajo fin de grado.

Si es una herramienta de análisis lo que has realizado, aquí puedes poner ejemplos de haberla utilizado para que se vea su utilidad.

Capítulo 7

Conclusiones

7.1. Consecución de objetivos

Esta sección es la sección espejo de las dos primeras del capítulo de objetivos, donde se planteaba el objetivo general y se elaboraban los específicos.

Es aquí donde hay que debatir qué se ha conseguido y qué no. Cuando algo no se ha conseguido, se ha de justificar, en términos de qué problemas se han encontrado y qué medidas se han tomado para mitigar esos problemas.

Y si has llegado hasta aquí, siempre es bueno pasarle el corrector ortográfico, que las erratas quedan fatal en la memoria final. Para eso, en Linux tenemos *aspell*, que se ejecuta de la siguiente manera desde la línea de *shell*:

```
aspell --lang=es_ES -c memoria.tex
```

7.2. Aplicación de lo aprendido

Aquí viene lo que has aprendido durante el Grado/Máster y que has aplicado en el TFG/TFM. Una buena idea es poner las asignaturas más relacionadas y comentar en un párrafo los conocimientos y habilidades puestos en práctica.

1. a

2. b

7.3. Lecciones aprendidas

Aquí viene lo que has aprendido en el Trabajo Fin de Grado/Máster.

1. Aquí viene uno.
2. Aquí viene otro.

7.4. Trabajos futuros

Ningún proyecto ni software se termina, así que aquí vienen ideas y funcionalidades que estaría bien tener implementadas en el futuro.

Es un apartado que sirve para dar ideas de cara a futuros TFGs/TFMs.

Apéndice A

Manual de usuario

A.1. Uso del componente `babiaxr`

El componente `babiaxr` permite añadir una interfaz interactiva de botones en un entorno de realidad virtual para controlar el mapa 3D generado por `babia-map`. Esta interfaz puede vincularse a las manos del usuario o posicionarse libremente en el entorno, permitiendo interacciones como zoom, cambio de capa y desplazamiento del mapa.

Propiedades generales

Propiedad	Tipo (por defecto)	Descripción
<code>handsUi</code>	boolean (true)	Habilita o deshabilita los botones en las manos VR.
<code>coordinates</code>	vec2	Coordenadas actuales del mapa apuntadas por la mano izquierda.

Transformaciones de grupos de botones

Propiedad	Tipo	Descripción
<code>entityRightButton</code>	string (JSON)	Posición y rotación del grupo de botones de la mano derecha.
<code>entityLeftButton</code>	string (JSON)	Posición y rotación del grupo de botones de la mano izquierda.
<code>crossEntity</code>	string (JSON)	Posición y rotación del grupo de botones direccionales.

Contenido de los grupos de botones

Propiedad	Tipo	Descripción
<code>rightButtons</code>	string (JSON array)	Botones de zoom.
<code>leftButtons</code>	string (JSON array)	Botones de cambio de capa.
<code>crossButtons</code>	string (JSON array)	Botones de desplazamiento (arriba, abajo, izquierda, derecha).
<code>coordinatesText</code>	string (JSON)	Configuración de estilo para el texto flotante de coordenadas.

Formato de cada botón

Cada botón se define como un objeto en formato JSON con los siguientes campos:

```
{
  icon": ,
  id": "zoomIn",
  "text": "Zoom +",
  .action": "zoomIn"
}
```

Descripción de campos:

- `icon`: (opcional) símbolo visual o carácter. No obligatorio.

- `id`: identificador único del botón.
- `text`: texto mostrado en el botón.
- `action`: acción que se ejecuta al pulsar el botón.

Ejemplo de configuración en HTML

A continuación se muestra un ejemplo de configuración del componente `babiaxr` dentro de una entidad `a-entity`:

```
<a-entity babia-map='
  handsUi: false;
  entityRightButton: {"position":"0.6 4 3", "rotation":"30 0 0"};
  entityLeftButton: {"position":"-1 3.7 3", "rotation":"30 0 0"};
  crossEntity: {"position":"-0.15 3.3 3", "rotation":"30 0 0"};
  rightButtons: [
    {"id":"zoomIn","text":"Zoom +","action":"zoomIn"},
    {"id":"zoomOut","text":"Zoom -","action":"zoomOut"}
  ];
  leftButtons: [
    {"id":"layerRelief","text":"Relieve","action":"elevation"},
    {"id":"layerSat","text":"Satélite","action":"satellite"}
  ];
  crossButtons: [
    {"id":"left","text":"Izquierda","action":"left"},
    {"id":"right","text":"Derecha","action":"right"},
    {"id":"up","text":"Arriba","action":"up"},
    {"id":"down","text":"Abajo","action":"down"}
  ];
  coordinatesText: {"position":"0 0.2 0","rotation":"-30 0 0",
    "scale":"0.3 0.3 0.3","color":"#ffffff","align":"center"}
'>
</a-entity>
```

Eventos disponibles

Los botones del componente `babiaxr` responden a los siguientes eventos de A-Frame:

Evento	Efecto
<code>click</code>	Ejecuta la acción asociada al botón.
<code>mouseenter</code>	Resalta el botón al pasar el puntero por encima.
<code>mouseleave</code>	Restaura el estilo del botón al salir el puntero.

Compatibilidad con manos y VR

Si la propiedad `handsUi` está activada, los grupos de botones se adjuntan automáticamente a las entidades con identificador `leftHand` y `rightHand`, que representan las manos del usuario en dispositivos de realidad virtual con seguimiento de manos. Los botones pueden activarse mediante punteros láser o interacción directa en el espacio tridimensional.

Bibliografía